Shee Seng Cheng 34612467
Tay Chee Hsian 34612513

FIT3143

Laboratory 01 Presentation

sshe0113@student.monash.edu
ctay0040@student.monash.edu

TASK 1

Serial Code - *Finding Prime Numbers*

Task 1

Serial Code

A Serial C program that finds all prime numbers strictly less than n, outputs them either to the terminal or a text file depending on n, and measures the execution time.

Approach

1st: Take an integer **n** from the user

```
//Ask user to input an integer
printf("Enter an integer number: ");
scanf("%d", &n); //Store the input to the address of 'n'
```

2nd: Check for every number k = 2 < n

```
// List all prime numbers until n
for (int k = 2; k < n; k++) {
```

Task 1

Serial Code

Approach

3rd: store detected prime into the boolean dynamically allocated array

```
// 2 is the only even prime number
if (k == 2) {
    primeArray[k] = true;
}
// Other even numbers are not prime
else if (k % 2 == 0) {
    continue;
}
```

```
// Store whether k is prime
primeArray[k] = isPrime;
```

Dynamic Memory Allocation

```
// Allocate memory to store primes
bool *primeArray = (bool *)calloc(n, sizeof(bool));
```

calloc() → allocates memory and initializes all to FALSE

Task 1

Serial Code

Approach

4th:

- Output:
- n < 100 → stdout
- n >= 100 → task1_output.txt
- Measure computation time and I/O time separately.

```
// Output result
if (n < 100) {
    printf("All prime numbers less than %d are:\n", n);
    // Serial loop to ensure sorted
    for (int k = 2; k < n; k++) {
        if (primeArray[k]){
            printf("%d\n", k);
        }
    }
} else {
    // Large n output to the file
    WritePrimesToFile("task1_output.txt", primes, count);
}
```

Task 1

Serial Code

Prime Checking (version 1)

- k represents the number currently being tested.
- We only test divisors from 2 to sqrt(k). Upper bond sqrt(k)
- Test every number range from 2 to sqrt(k)
- If a divisor found a earlier loop exists is ensured

This is inefficiency as we know 2 is the only even prime number we can used that as a hint to further optimize our task1

```
// List all prime numbers until n
for (int k = 2; k < n; k++) {
    // Boolean variable
    int is_prime = 1; //1 True, 0 False

    // Check from 2 until sqrt k
    for (int i = 2; i <= sqrt(k); i++) {

        //if k has a divisor then k not a prime
        if (k % i == 0){
            is_prime = 0;
            break;
        }

    }

    if (is_prime){
        //Print formatted data to a specific output stream.
        fprintf(output, "%d\n", k);
    }

}
```

Task 1

Serial Code

Prime Checking (optimize version)

- k represents the number currently being tested.
- We only test divisors from 2 to sqrt(k). Upper bond sqrt(k)
- Odd only step, 2 is only even prime number, after that skip all even candidates and odd divisor check
- If a divisor found a earlier loop exists is ensured

If k has a factor larger than sqrt(k), it must have a corresponding factor smaller than sqrt(k). Therefore, checking beyond sqrt(k) is unnecessary.

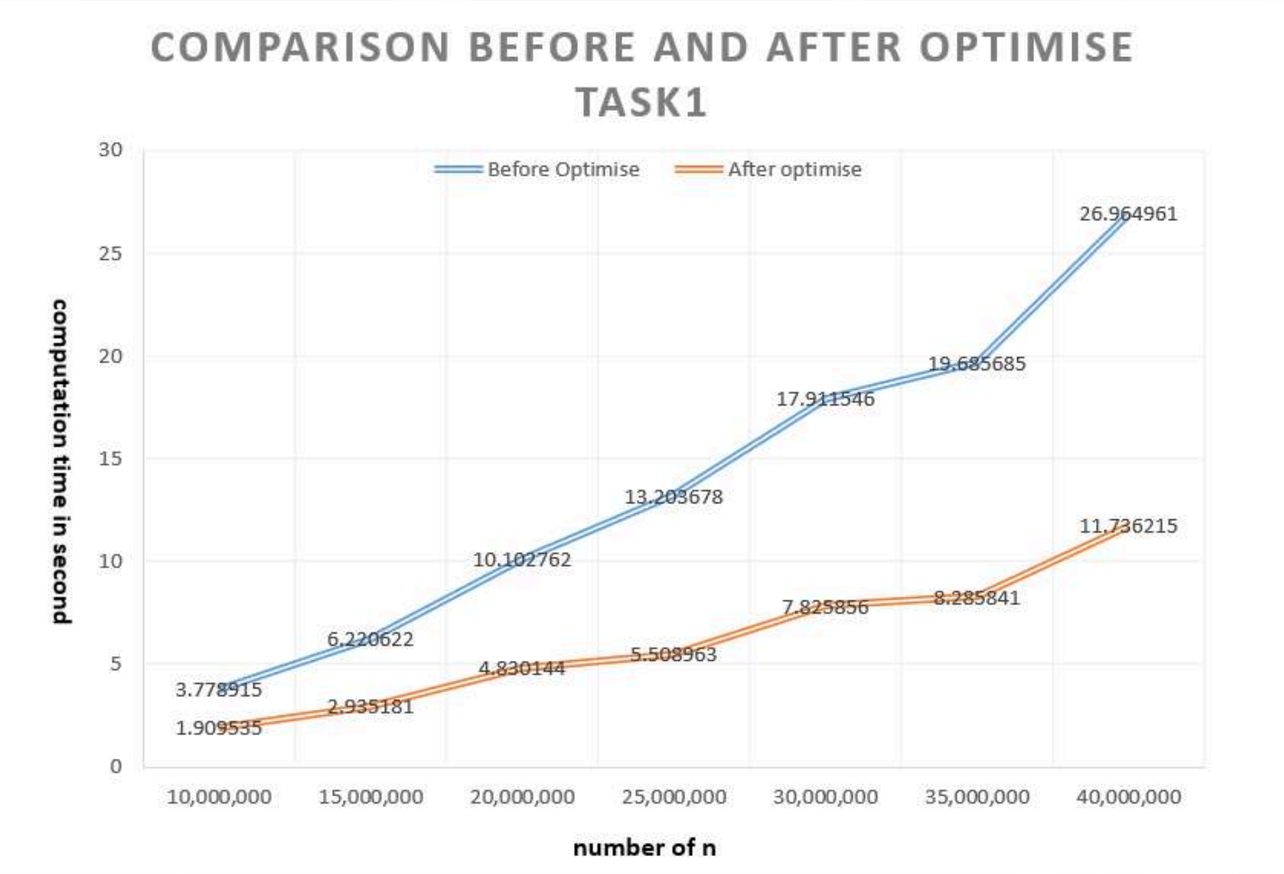
```
// List all prime numbers until n
for (int k = 2; k < n; k++) {

    // 2 is the only even prime number
    if (k == 2) {
        primeArray[k] = true;
    }
    // Other even numbers are not prime
    else if (k % 2 == 0) {
        continue;
    }
    // Only check odd numbers
    else {
        bool isPrime = true;
        // Check from 3 until sqrt(k), skipping even numbers
        int range = (int)sqrt(k);

        for (int i = 3; i <= range; i += 2) {
            // If k has a divisor, k is not prime
            if (k % i == 0) {
                isPrime = false;
                break;
            }
        }
        // Store whether k is prime
        primeArray[k] = isPrime;
    }
}
```

Task 1

Serial Code



- We can clearly see that the computation time is reduced significantly, the larger the n value the higher the difference between before and after optimization
- Speed up (approximately): taking n = 40,000,000 we got around speed up factor of 2.30 compare to unoptimized baseline.

Task 1

Serial Code

Timing

- CLOCK_MONOTONIC gives a clock that continuously moves forward.
- start records the beginning.
- end records the end.
- Difference between them gives computation time.

```
struct timespec start, end, startW, endW;
double timetaken, time_write;
```

```
//Start timing only the computation
// Get current clock time. (Monotonic = always move foward)
clock_gettime(CLOCK_MONOTONIC, &start);
```

```
// Get current clock time (end for computation)
clock_gettime(CLOCK_MONOTONIC, &end);

// Duration of the computation process
timetaken = (end.tv_sec - start.tv_sec) * 1e9; //* 1e9 to nanoseconds
//include nano seconds
timetaken = (timetaken + (end.tv_nsec - start.tv_nsec)) / 1e9; // turn into seconds
```

We separate computation time from I/O time so that file or terminal output does not distort the measurement of the prime-search computation.

```
clock_gettime(CLOCK_MONOTONIC, &startW);
// Output result
if (n < 100) {
```

```
clock_gettime(CLOCK_MONOTONIC, &endW);
// Duration of the computation process
time_write = (endW.tv_sec - startW.tv_sec) * 1e9; //* 1e9 to nanoseconds
//include nano seconds
time_write = (time_write + (endW.tv_nsec - startW.tv_nsec)) / 1e9; // turn into seconds
```

Task 1

Serial Code

Sample output

- Below 100

```
$ ./task1
Enter an integer number: 99
Time taken to compute: 0.000039 seconds
All prime numbers less than 99 are:
2
3
5
7
11
13
17
19
23
29
31
37
41
43
47
53
59
61
67
71
73
79
83
89
97
Time taken to write: 0.000046 seconds
Overall time taken: 0.000084 seconds
```

- More than or equal to 100

```
$ ./task1
Enter an integer number: 200
Time taken to compute: 0.000007 seconds
Result has been written to task1_output.txt
Time taken to write: 0.007803 seconds
Overall time taken: 0.007810 seconds
```

```
$ cat task1_output.txt
2
3
5
7
11
13
17
19
23
29
31
37
41
43
47
53
59
61
67
71
73
79
83
89
97
101
103
107
109
113
127
131
137
139
149
151
157
163
167
173
179
181
191
193
197
199
```

TASK 2

POSIX Threads - *Finding Prime Numbers*

Task 2

POSIX Threads

```c
void *find_prime(void *arg)
{
    int thread_id = *(int *)arg;

    // Instead of giving each thread one continuous chunk of words,
    // distribute the work by taking every num_threads-th item.
    for (int k = 2 + thread_id; k < n; k += num_threads) {
        if (k == 2) {  // 2 is the only even prime number
            prime_array[k] = true;
        } else if (k % 2 != 0) { // Handles the case when k is odd number
            bool is_prime = true;
            // Only needs to check whether it is divisible by any integer between 2 and sqrt(k)
            int range = (int)sqrt(k);
            for (int i = 3; i <= range; i += 2) {  // Jump 2 steps to save time
                if (k % i == 0) {
                    is_prime = false;
                    break;
                }
            }
            if (is_prime) {
                prime_array[k] = true;
            }
        }
    }
    pthread_exit(NULL);  // End thread
}
```

- **Cyclic Partitioning:** Threads distribute workload by taking strides equal to the number of threads
- **Natural Load Balancing:** Evenly mixes computationally small and large numbers among all threads
- **Lock-Free Design:** Eliminates Mutex overhead, preventing thread contention and ensuring near-linear speedup

Task 2

POSIX Threads

```
bool *prime_array;  // A boolean array to record prime numbers
```

```
prime_array = (bool *)calloc(n, sizeof(bool));
```

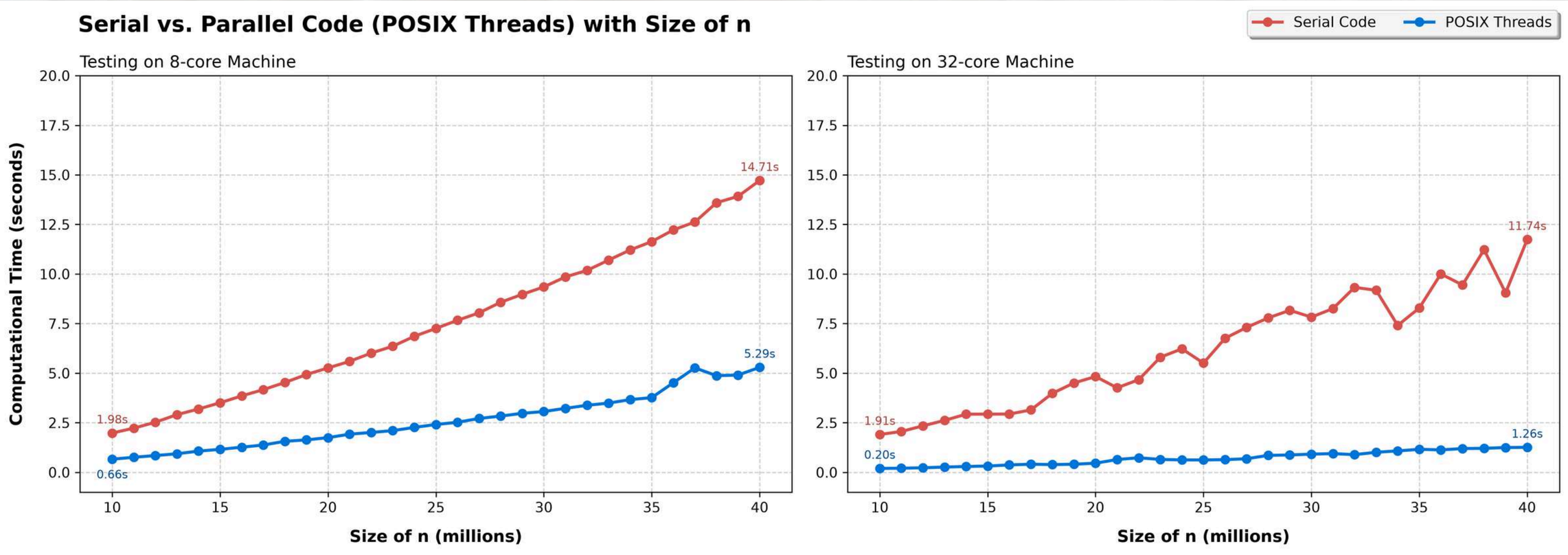
```
for (int k = 2 + thread_id; k < n; k += num_threads)
{
    if (k == 2)  // 2 is the only even prime number
    {
        prime_array[k] = true;
    }
```

- **Zero Race Condition:** Threads never attempt to write to the same memory address simultaneously, the program runs safely at maximum speed without the need for Mutex locks.

- **Global Boolean Map:** A single, lightweight Boolean array (prime_array) is allocated in the heap and shared across all threads to record prime numbers.
- **Independent Indexing:** Using Cyclic Partitioning, each thread calculates and writes to strictly unique indices.

Task 2

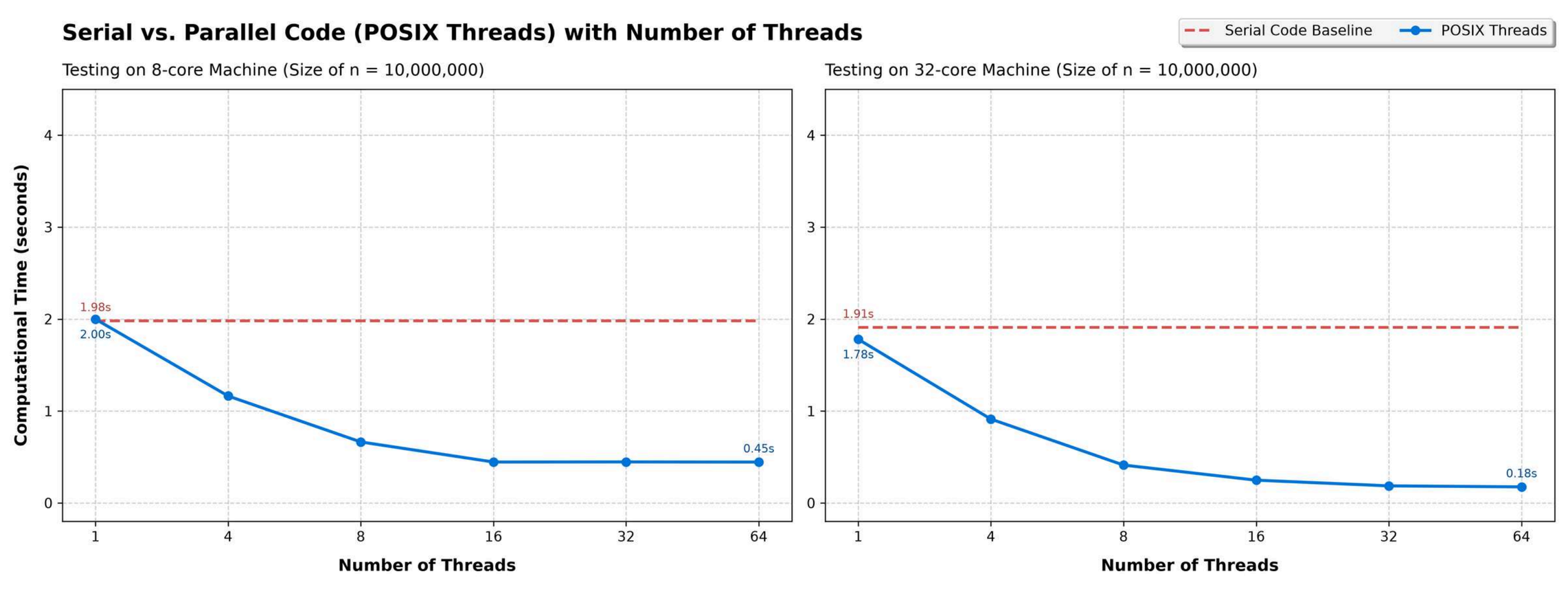
POSIX Threads



- POSIX Threads drastically reduce computation time compared to the serial code across the architectures.
- As the size of n scales up to 40 million, the serial execution time climbs rapidly , whereas the POSIX Threads computation remains consistently low and efficient.

Task 2

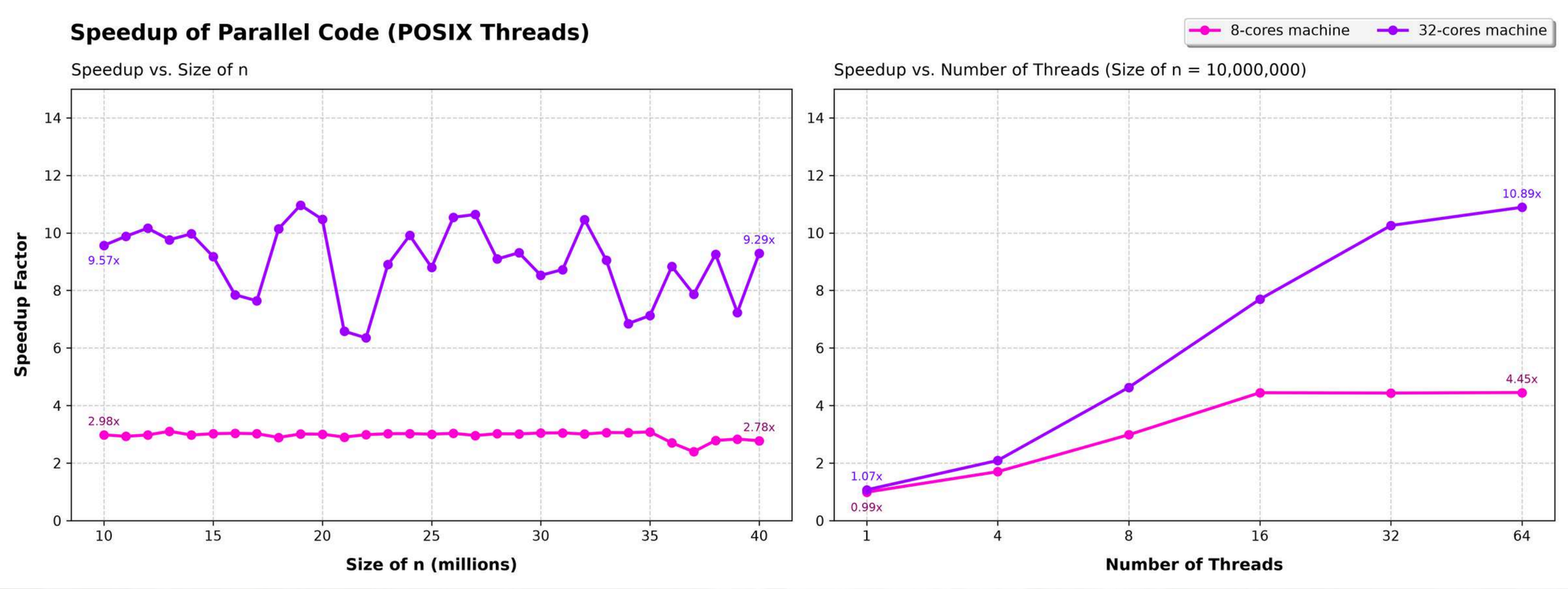
POSIX Threads



- Execution time plateaus after 16 threads, clearly demonstrating the overhead of thread over-subscription and context switching.
- Computation time drops continuously, reaching a remarkable 0.18s at 64 threads, proving excellent utilization of high-core-count processors.

Task 2

POSIX Threads



Task 2

POSIX Threads

Speedup vs. Size of n

- **8-Core:** The speedup remains highly stable at around **3x**. This demonstrates consistent parallel efficiency, though it is physically limited by the system's 8 hardware threads, memory bandwidth, and the optimized serial baseline.
- **32-core:** The speedup **fluctuates between 6x and 11x**. This higher overall performance proves the scalability of POSIX threads on high-core processors. The visible fluctuations are expected behavior due to real-world **Non-Uniform Memory Access (NUMA)** latency and **OS thread scheduling**.

Speedup vs. Number of Threads

- **Initial Scaling:** Adding more threads effectively divides the workload, resulting in a proportional and rapid increase in processing speed.
- **Hardware Saturation:** Once the number of threads exceeds the machine's actual physical cores, performance completely plateaus. The CPU reaches its maximum capacity and begins processing power on switching between tasks.

TASK 3

Open MP - *Finding Prime Numbers*

Task 3

OpenMP

```
// Dynamic scheduling with a chunk size of 500 to minimize scheduling overhead
#pragma omp parallel for num_threads(num_threads) default(none) shared(primeArray, n) schedule(dynamic, 500)
for (int k = 2; k < n; k++) {

    // 2 is the only even prime number
    if (k == 2) {
        primeArray[k] = true;
    }
    // Other even numbers are not prime
    else if (k % 2 == 0) {
        continue;
    }
    // Only check odd numbers
    else {
        bool isPrime = true;
        // Check from 3 until sqrt(k), skipping even numbers
        int range = (int)sqrt(k);

        for (int i = 3; i <= range; i += 2) {
            // If k has a divisor, k is not prime
            if (k % i == 0) {
                isPrime = false;
                break;
            }
        }
        // Store whether k is prime
        primeArray[k] = isPrime;
    }
}
```

- **Data Scoping (default(none)):** Disables OpenMP's implicit variable scoping, enforcing compile-time validation to prevent unintended variable sharing and race hazards.
- **Shared Data (shared(primeArray, n)):** Both the input bound n and the boolean flag array (primeArray) reside in shared memory for direct access by all threads.
- Loop index k, inner iteration variable i, isPrime, and range are allocated on each thread's private stack

Task 3

OpenMP

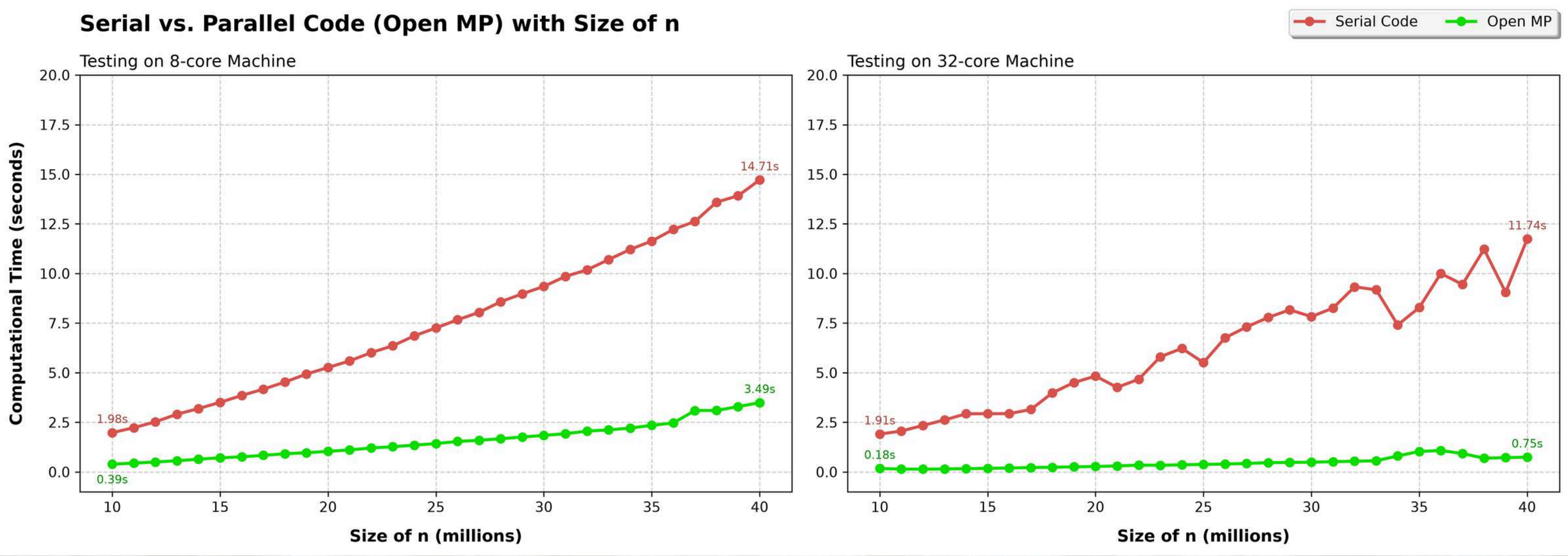
```
// Dynamic scheduling with a chunk size of 500 to minimize scheduling overhead
#pragma omp parallel for num_threads(num_threads) default(none) shared(primeArray, n) schedule(dynamic, 500)
for (int k = 2; k < n; k++) {
```

- Prime checking is computationally unbalanced since higher numbers require more factor tests up to sqrt(k)
- making the upper range significantly more CPU-intensive than the lower range.
- If we used static scheduling, threads handling lower numbers would finish early and sit idle, degrading our overall parallel speedup.

- **Solution:** Dynamic scheduling resolves this by assigning chunks of iterations to threads at runtime as they become available (500 chunk size is the optimal for our hardware)

Task 3

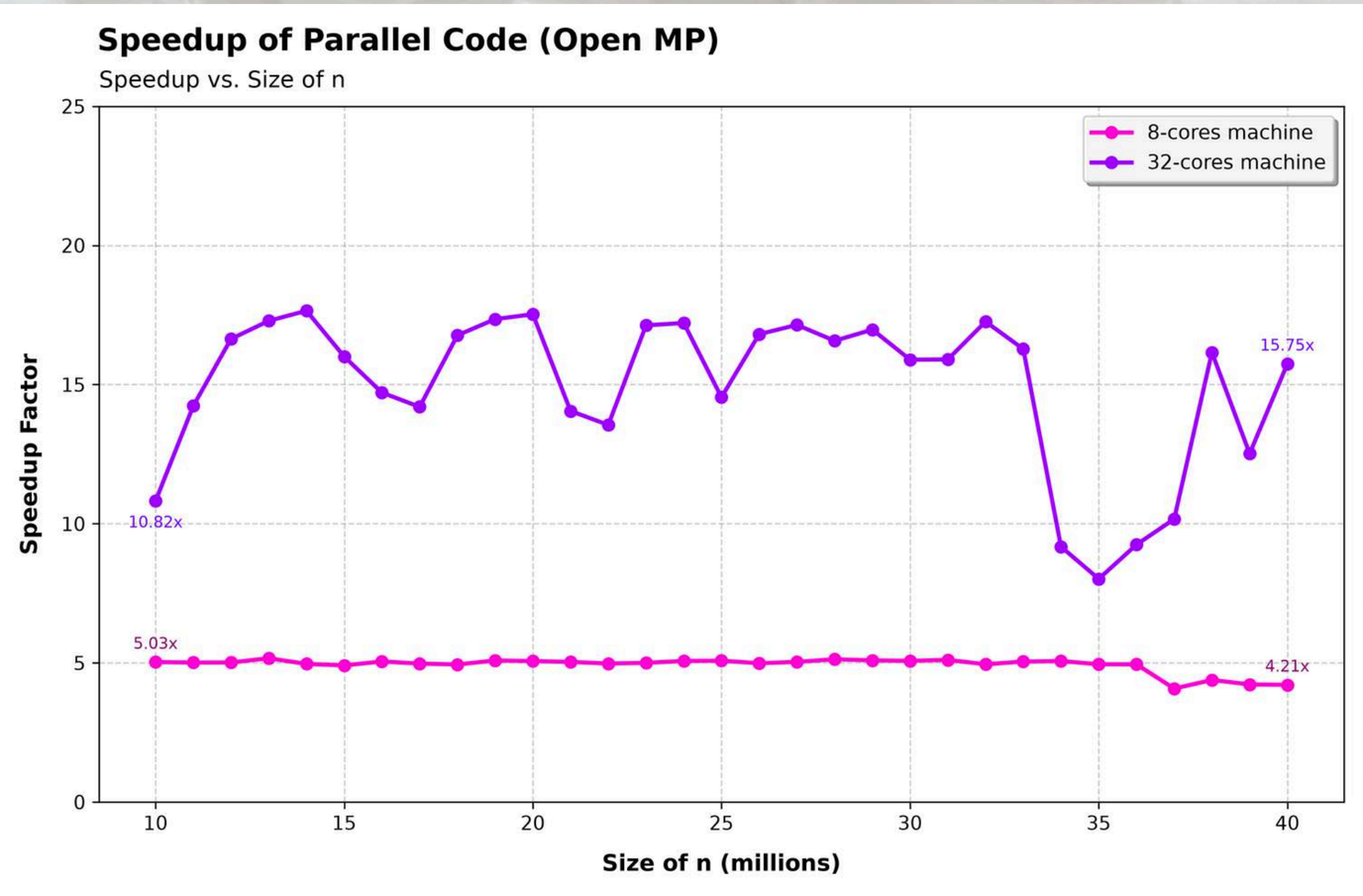
Open MP



- Open MP maintains flat execution times across both platforms, rising gradually to only 3.49 seconds on the 8-core machine and 0.75 seconds on the 32-core machine when n is 40 million.
- The schedule(dynamic, 500) directive prevents thread stalls on large prime checks, outperforming the steep execution climb seen in the serial baseline.

Task 3

Open MP

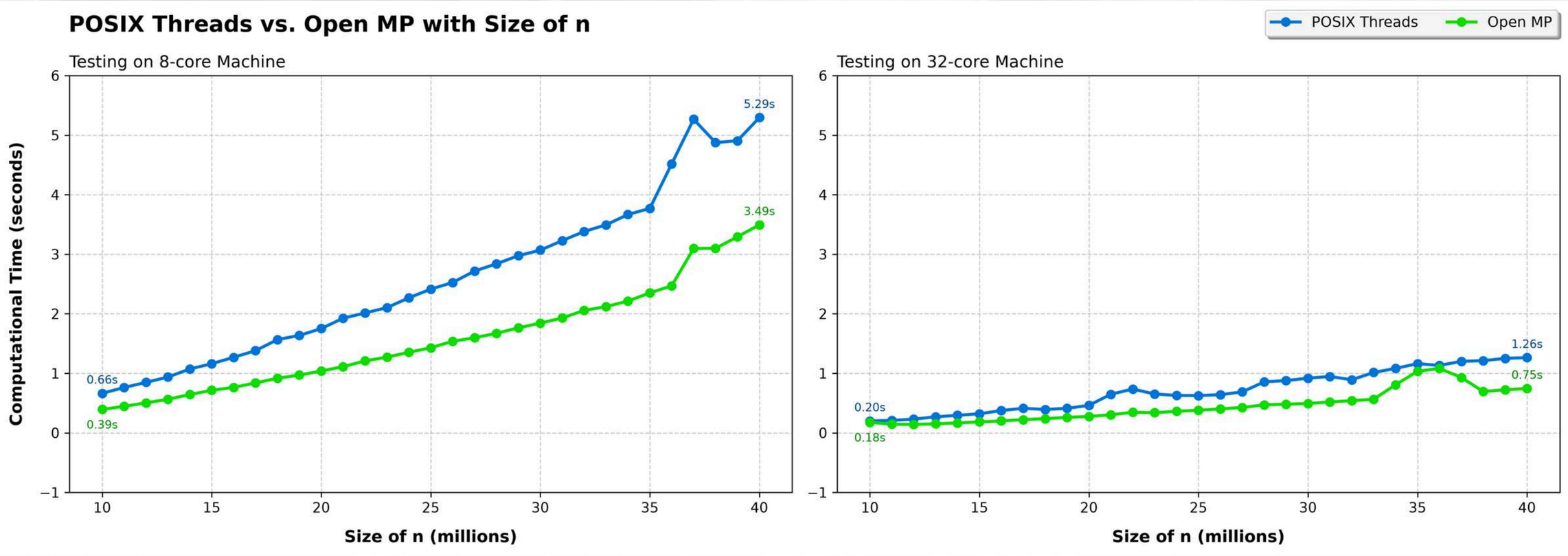


- **8-core machine** achieves a consistent speedup around **4.2x to 5.0x**, reflecting high parallel efficiency bounded strictly by 8 physical processing cores.
- **32-core machine** scales dramatically to **15.75x**, representing a ~50% gain over POSIX Threads by eliminating false sharing and improving memory chunking.

- Speedup fluctuations on the 32-core machine illustrate runtime scheduling overhead and NUMA memory latency when distributing chunks across high thread counts.

Task 3

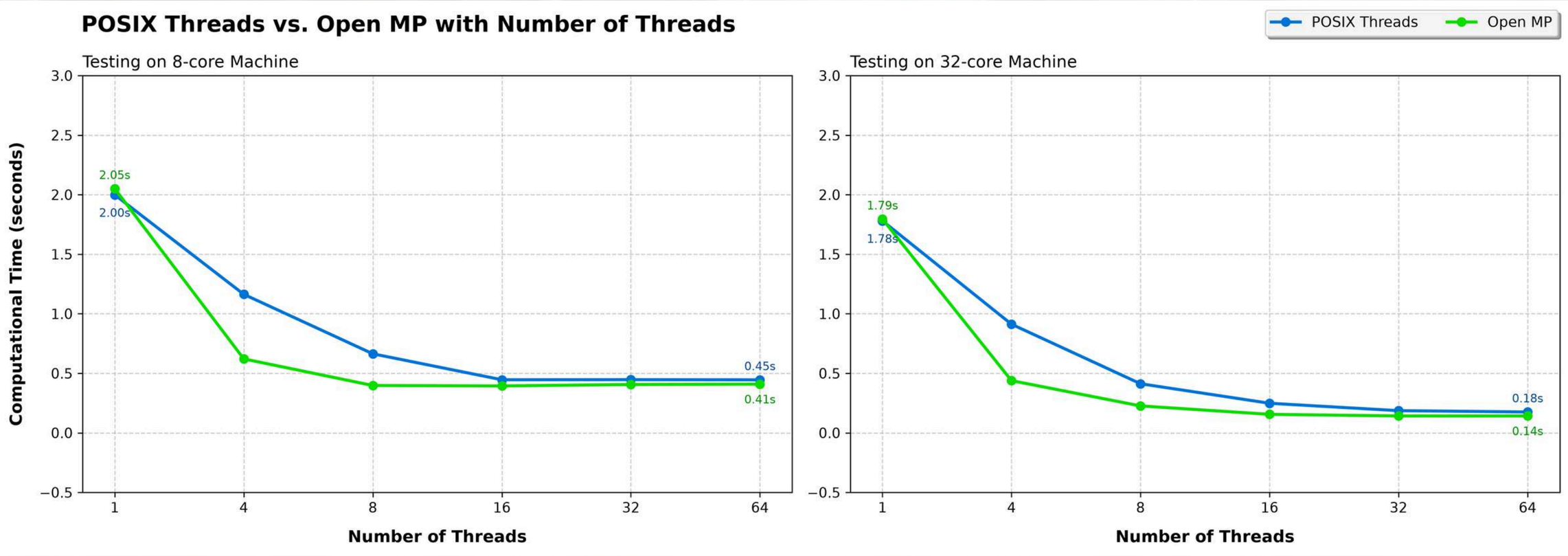
Open MP



- Open MP consistently processes each scale n faster than POSIX Threads on both machine configurations.
- Open MP's batch chunking (chunk = 500) eliminates the cache line conflicts (false sharing) inherent in stride-1 cyclic partitioning.

Task 3

Open MP



- At 1 thread, Open MP takes a bit longer than POSIX Threads due to the internal Open MP runtime and team initialization overhead.
- Across 4 to 64 threads, Open MP reaches peak throughput faster, achieving optimal execution.

Conclusion

Summary of Finding

- **Parallel Efficiency:** Both POSIX Threads and OpenMP achieved substantial, near-linear speedups over the serial baseline (cutting computation time from ~11.8s down to <1s at 40M). Serial vs OpenMP

- **Framework Comparison:** OpenMP (dynamic, 500) proved superior to cyclic Pthreads across all problem sizes by resolving **cache-line contention** (false sharing) and non-uniform work distribution.
  - In cyclic Pthreads, adjacent threads write to neighboring indices that share the same 64-byte CPU cache line, causing cache-invalidation overhead (false sharing). In contrast, OpenMP's 500-iteration chunks give each core its own continuous memory block, maximizing cache efficiency and eliminating cache-line contention

Major Bottlenecks & Hardware Limitations Identified

- **Thread Over-Subscription:** Scaling beyond physical core counts (e.g. 16 threads run on an 8-core CPU) caused speedup saturation due to OS context-switching overhead. SERIAL vs POSIX
  - Every time the OS swaps a thread, it must perform a context switch (saving the active thread's registers, stack pointers, and program counter, and loading the next thread's state). The swap itself introduces CPU overhead.
- We clearly see that after number of thread > CPU Core the performance does not significantly increase (plateaus).

FUTURE WORK & RECOMMENDATION

- Automatically scale OpenMP chunk sizes based on the CPU core count and problem size n, rather than using a static chunk size of 500.
- Implementing thread pinning will bind worker threads to specific physical CPU cores. This stops the operating system scheduler from moving threads around, further reducing cache-miss penalties

Q & A

Thank You